

CMPE 372 — Assignment 2

Data Modeling + UI–Database Relationship

Project: Rukn (رُكن) — Your Cozy Reading & Journaling Corner **Course:** CMPE 372 — User Interface Design (Section 02) **Deadline:** April 26, 2026

Weight: 10%

Team Members

Name	ID
Zomorodah Bakhit	122200145
Omar Othman	122200162
Rewan Afifi	122200013
Reem Khalil	122200137
Faysal Dabbagh	122200015
Omar Anouti	121200031

Project Overview

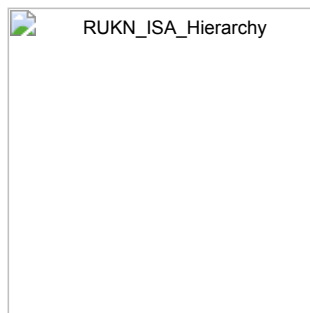
Rukn is a bilingual (Arabic/English) web application that unifies three reading experiences into one space: a full in-app e-book reader for PDF and EPUB files, a passage-linked journaling system, and a customizable cozy virtual environment with ambient sounds, themes, and a focus timer. A user can select a passage while reading, attach a journal entry directly to that passage, or write freeform thoughts — all without leaving the app. Users are divided into three conceptual roles: **guests** (unregistered visitors with time-limited sampling), **free registered users** (full reading, journaling, and library tools), and **paid registered users** (offline downloads, full customization, ad-free experience).

This document presents the data layer that supports this system: the entities, their attributes, their relationships, the resulting relational schema, its normalization, and the end-to-end mapping between the interface and the database.

A. Conceptual Data Model

A.1 User Hierarchy (ISA / Specialization)

Rukn uses a specialization hierarchy to model its three user types. A `User` is an abstract superclass; `Guest` and `Registered` are its two subclasses; `Registered` is itself a superclass with `Free User` and `Paid User` as subclasses.



This is implemented via **class-table inheritance**: a base `users` table holds attributes shared by all registered users, while `free_users` and `paid_users` each hold the `user_id` as both primary key and foreign key (currently placeholder rows, extensible for future tier-specific attributes like subscription periods). `Guest` is conceptual only and has no table — guest sessions are enforced client-side through browser storage.

A.2 Entities and Attributes

The data model consists of **ten entities** plus **two junction tables** for N:M relationships.

1. **User** — base table for all registered users.

Attribute	Type	Notes
user_id	INT	PK
username	VARCHAR(50)	UNIQUE
email	VARCHAR(100)	UNIQUE
password_hash	VARCHAR(255)	
preferred_lang	ENUM('AR','EN')	
created_at	TIMESTAMP	
last_login	TIMESTAMP	

2. **FreeUser** — subclass of User.

Attribute	Type	Notes
user_id	INT	PK, FK → User

3. **PaidUser** — subclass of User.

Attribute	Type	Notes
user_id	INT	PK, FK → User
subscribed_at	TIMESTAMP	When they upgraded

4. **UserPreferences** — 1:1 with User, auto-created on signup.

Attribute	Type	Notes
preference_id	INT	PK
user_id	INT	FK, UNIQUE
theme	VARCHAR(50)	
background	VARCHAR(100)	
font	VARCHAR(50)	
font_size	INT	
ambient_sound	VARCHAR(50)	
sound_volume	DECIMAL(3,2)	0.00–1.00

5. **Book** — uploaded or public-domain reading material.

Attribute	Type	Notes
book_id	INT	PK
title	VARCHAR(255)	
author	VARCHAR(255)	
format	ENUM('PDF','EPUB')	
file_path	VARCHAR(500)	
cover_image	VARCHAR(500)	
total_pages	INT	
is_public_domain	BOOLEAN	
uploaded_by	INT	FK → User (NULL for public domain)
uploaded_at	TIMESTAMP	

6. **ReadingProgress** — per-user-per-book reading state.

Attribute	Type	Notes
progress_id	INT	PK
user_id	INT	FK
book_id	INT	FK
current_page	INT	
pages_read	INT	

Attribute	Type	Notes
total_seconds_read	INT	
last_read_at	TIMESTAMP	
is_completed	BOOLEAN	

UNIQUE(user_id, book_id) — exactly one progress record per user per book.

7. Highlight — a selected passage on a page.

Attribute	Type	Notes
highlight_id	INT	PK
user_id	INT	FK
book_id	INT	FK
page_number	INT	
selected_text	TEXT	The highlighted passage
color	ENUM('yellow','green','blue','pink')	
created_at	TIMESTAMP	

8. Bookmark — a saved page location, no text.

Attribute	Type	Notes
bookmark_id	INT	PK
user_id	INT	FK
book_id	INT	FK
page_number	INT	
created_at	TIMESTAMP	

UNIQUE(user_id, book_id, page_number).

9. Journal — container for journal entries. One freeform per user (auto-created on signup), one book journal per user per book (auto-created on first entry for that book).

Attribute	Type	Notes
journal_id	INT	PK
user_id	INT	FK
book_id	INT	FK, NULL for freeform
type	ENUM('book','freeform')	
title	VARCHAR(255)	User-editable
created_at	TIMESTAMP	

UNIQUE(user_id, book_id) — prevents duplicate book journals.

10. JournalEntry — an individual entry inside a journal. Optionally linked to a highlight.

Attribute	Type	Notes
entry_id	INT	PK
journal_id	INT	FK
highlight_id	INT	FK, NULL if not linked to a passage
title	VARCHAR(255)	
content	TEXT	
entry_date	DATE	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

11. Tag — a user's reusable label for journal entries (Obsidian-style).

Attribute	Type	Notes
tag_id	INT	PK
user_id	INT	FK
name	VARCHAR(50)	

UNIQUE(user_id, name).

12. Collection — a user-defined folder for books.

Attribute	Type	Notes
collection_id	INT	PK
user_id	INT	FK
name	VARCHAR(100)	
created_at	TIMESTAMP	

UNIQUE(user_id, name).

Junction tables

JournalEntryTag — models the N:M between entries and tags.

Attribute	Type	Notes
entry_id	INT	PK, FK → JournalEntry
tag_id	INT	PK, FK → Tag

CollectionBook — models the N:M between collections and books.

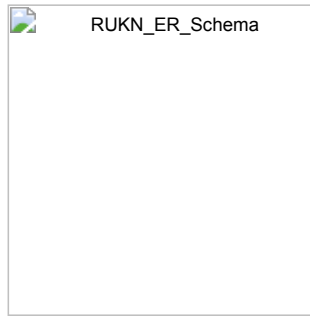
Attribute	Type	Notes
collection_id	INT	PK, FK → Collection
book_id	INT	PK, FK → Book

A.3 Relationships

Relationship	Cardinality	Notes
User ↔ FreeUser	1 : (0,1)	ISA specialization
User ↔ PaidUser	1 : (0,1)	ISA specialization
User → UserPreferences	1 : 1	Auto-created on signup
User → Book	1 : N	A user uploads 0+ books; a book has at most one uploader
User → ReadingProgress	1 : N	
Book → ReadingProgress	1 : N	
User → Highlight	1 : N	
Book → Highlight	1 : N	
User → Bookmark	1 : N	
Book → Bookmark	1 : N	
User → Journal	1 : N	Every user has ≥ 1 journal (freeform)
Book → Journal	1 : N	Optional — only books that users have journaled about
Journal → JournalEntry	1 : N	
Highlight → JournalEntry	1 : N	Optional link; entries don't require a highlight
User → Tag	1 : N	Tags belong to a user
JournalEntry ↔ Tag	N : M	Via JournalEntryTag
User → Collection	1 : N	
Collection ↔ Book	N : M	Via CollectionBook

B. ER Diagram

The complete relational schema with crow's-foot cardinality notation:



Reading the diagram:

- **Each entity box** shows the table name (brown header) and its attributes.
- **PK, FK, UK** badges mark primary keys, foreign keys, and unique keys.
- **Junction tables** (orange headers — `COLLECTION_BOOK`, `JOURNAL_ENTRY_TAG`) implement the N:M relationships.
- **Crow's-foot endings** on edges indicate cardinality: a "crow's foot" (<) means "many," a single bar (|) means "exactly one," and a circle (o) means "zero (optional)." So o< reads as "zero or many" and || as "exactly one."

C. Relational Schema

Every entity from Section A.2 maps to a table below. Primary keys are marked `PRIMARY KEY`. Foreign keys use `ON DELETE` behavior that matches the intended semantics — cascade when the data is owned by a user, set null when the reference can be broken without losing the child row.

```

-- =====
-- USERS (base table for all registered users)
-- =====

CREATE TABLE users (
    user_id          INT          PRIMARY KEY AUTO_INCREMENT,
    username         VARCHAR(50)  NOT NULL UNIQUE,
    email            VARCHAR(100)  NOT NULL UNIQUE,
    password_hash     VARCHAR(255) NOT NULL,
    preferred_lang    ENUM('AR','EN') NOT NULL DEFAULT 'EN',
    created_at        TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    last_login        TIMESTAMP    NULL
);

-- =====
-- FREE_USERS (ISA subclass - placeholder for future free-tier attrs)
-- =====

CREATE TABLE free_users (
    user_id          INT          PRIMARY KEY,
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);

-- =====
-- PAID_USERS (ISA subclass - extensible for subscription attributes)
-- =====

CREATE TABLE paid_users (
    user_id          INT          PRIMARY KEY,
    subscribed_at     TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);

-- =====
-- USER_PREFERENCES (1:1 with users, auto-created on signup)
-- =====

CREATE TABLE user_preferences (
    preference_id     INT          PRIMARY KEY AUTO_INCREMENT,
    user_id           INT          NOT NULL UNIQUE,
    theme             VARCHAR(50)  DEFAULT 'default',
    background        VARCHAR(100) DEFAULT 'plain',
    font              VARCHAR(50)  DEFAULT 'Lora',
    font_size         INT          DEFAULT 16,
    ambient_sound     VARCHAR(50)  DEFAULT 'none',
    sound_volume       DECIMAL(3,2) DEFAULT 0.50,
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);

-- =====
-- BOOKS
-- =====

CREATE TABLE books (
    book_id          INT          PRIMARY KEY AUTO_INCREMENT,
    title            VARCHAR(255)  NOT NULL,
    author           VARCHAR(255),
    format           ENUM('PDF','EPUB') NOT NULL,
    file_path        VARCHAR(500)  NOT NULL,
    cover_image      VARCHAR(500),
    total_pages       INT          NOT NULL,

```

```

    is_public_domain    BOOLEAN          NOT NULL DEFAULT FALSE,
    uploaded_by         INT              NULL,
    uploaded_at         TIMESTAMP        DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (uploaded_by) REFERENCES users(user_id) ON DELETE SET NULL
);

-- =====
-- READING_PROGRESS (one row per user per book)
-- =====

CREATE TABLE reading_progress (
    progress_id         INT              PRIMARY KEY AUTO_INCREMENT,
    user_id             INT              NOT NULL,
    book_id             INT              NOT NULL,
    current_page        INT              NOT NULL DEFAULT 1,
    pages_read          INT              NOT NULL DEFAULT 0,
    total_seconds_read  INT              NOT NULL DEFAULT 0,
    last_read_at        TIMESTAMP        DEFAULT CURRENT_TIMESTAMP,
    is_completed        BOOLEAN          NOT NULL DEFAULT FALSE,
    UNIQUE KEY uq_progress_user_book (user_id, book_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (book_id) REFERENCES books(book_id) ON DELETE CASCADE
);

-- =====
-- HIGHLIGHTS (selected passages; notes are separate journal entries)
-- =====

CREATE TABLE highlights (
    highlight_id        INT              PRIMARY KEY AUTO_INCREMENT,
    user_id             INT              NOT NULL,
    book_id             INT              NOT NULL,
    page_number         INT              NOT NULL,
    selected_text        TEXT            NOT NULL,
    color               ENUM('yellow','green','blue','pink') NOT NULL DEFAULT 'yellow',
    created_at          TIMESTAMP        DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (book_id) REFERENCES books(book_id) ON DELETE CASCADE
);

-- =====
-- BOOKMARKS (page markers only, no text)
-- =====

CREATE TABLE bookmarks (
    bookmark_id         INT              PRIMARY KEY AUTO_INCREMENT,
    user_id             INT              NOT NULL,
    book_id             INT              NOT NULL,
    page_number         INT              NOT NULL,
    created_at          TIMESTAMP        DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_bookmark (user_id, book_id, page_number),
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (book_id) REFERENCES books(book_id) ON DELETE CASCADE
);

-- =====
-- JOURNALS
-- type='book'      → book_id NOT NULL, auto-created on first entry
-- type='freeform' → book_id NULL, exactly one per user, auto-created on signup

```

```

-- =====
CREATE TABLE journals (
    journal_id      INT          PRIMARY KEY AUTO_INCREMENT,
    user_id         INT          NOT NULL,
    book_id         INT          NULL,
    type            ENUM('book','freeform') NOT NULL,
    title           VARCHAR(255),
    created_at      TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_journal_user_book (user_id, book_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (book_id) REFERENCES books(book_id) ON DELETE CASCADE
);

-- =====
-- JOURNAL_ENTRIES (the actual written thoughts)
-- =====

CREATE TABLE journal_entries (
    entry_id        INT          PRIMARY KEY AUTO_INCREMENT,
    journal_id      INT          NOT NULL,
    highlight_id    INT          NULL,
    title           VARCHAR(255),
    content         TEXT         NOT NULL,
    entry_date      DATE         NOT NULL DEFAULT (CURRENT_DATE),
    created_at      TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
    updated_at      TIMESTAMP     DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (journal_id) REFERENCES journals(journal_id) ON DELETE CASCADE,
    FOREIGN KEY (highlight_id) REFERENCES highlights(highlight_id) ON DELETE SET NULL
);

-- =====
-- TAGS (user's private Obsidian-style vocabulary)
-- =====

CREATE TABLE tags (
    tag_id          INT          PRIMARY KEY AUTO_INCREMENT,
    user_id         INT          NOT NULL,
    name            VARCHAR(50)  NOT NULL,
    UNIQUE KEY uq_tag_user_name (user_id, name),
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);

-- =====
-- JOURNAL_ENTRY_TAGS (N:M junction)
-- =====

CREATE TABLE journal_entry_tags (
    entry_id        INT          NOT NULL,
    tag_id          INT          NOT NULL,
    PRIMARY KEY (entry_id, tag_id),
    FOREIGN KEY (entry_id) REFERENCES journal_entries(entry_id) ON DELETE CASCADE,
    FOREIGN KEY (tag_id) REFERENCES tags(tag_id) ON DELETE CASCADE
);

-- =====
-- COLLECTIONS (user-defined book folders)
-- =====

CREATE TABLE collections (
    collection_id   INT          PRIMARY KEY AUTO_INCREMENT,

```



```

    user_id          INT                NOT NULL,
    name             VARCHAR(100)       NOT NULL,
    created_at       TIMESTAMP          DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_collection_name (user_id, name),
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);

-- =====
-- COLLECTION_BOOKS (N:M junction)
-- =====

CREATE TABLE collection_books (
    collection_id    INT                NOT NULL,
    book_id          INT                NOT NULL,
    PRIMARY KEY (collection_id, book_id),
    FOREIGN KEY (collection_id) REFERENCES collections(collection_id) ON DELETE CASCADE,
    FOREIGN KEY (book_id) REFERENCES books(book_id) ON DELETE CASCADE
);

```

D. Normalization Logic

The schema satisfies **Third Normal Form (3NF)**. Each level is demonstrated below with concrete references to the tables above.

D.1 First Normal Form (1NF)

Every column holds a single atomic value. No table contains lists, arrays, or repeating groups.

- A journal entry's tags are **not** stored as "wisdom, empathy, growth" inside `journal_entries`. They live in the `journal_entry_tags` junction table, one pair per row.
- A collection's books are **not** stored as a list of IDs inside `collections`. They live in `collection_books`, one pair per row.
- A book's author is stored as a single string field (multi-author support is not in the base scope).

Every table has a defined primary key, so every row is uniquely identifiable.

D.2 Second Normal Form (2NF)

2NF is only at risk in tables with **composite primary keys**. The schema has two:

- `journal_entry_tags` (`entry_id`, `tag_id`) — contains no non-key attributes, so no partial dependency can exist. Trivially 2NF.
- `collection_books` (`collection_id`, `book_id`) — same: no non-key attributes. Trivially 2NF.

Every other table uses a single-column surrogate primary key (`_id`), so 2NF is automatically satisfied. Notably, `reading_progress` uses a surrogate `progress_id` rather than the composite (`user_id`, `book_id`) — we enforce uniqueness via a separate `UNIQUE` constraint. This keeps foreign key references simple (a single column) and avoids any partial-dependency concerns entirely.

D.3 Third Normal Form (3NF)

No non-key attribute depends on another non-key attribute. Every fact lives in exactly one table:

- **User preferences** (`theme`, `font`, `ambient_sound`) are separated into `user_preferences`, not stored on `users`. Putting them on `users` would create a transitive dependency chain (`user_id` → `preferences_group` → `theme`). The dedicated table breaks that chain.
- **Book metadata** (`title`, `author`, `total_pages`) lives only in `books`. When a journal references a book, it references `book_id`, never a copy of the title. Renaming a book updates one row.
- **Tag names** live only in `tags`. `journal_entry_tags` stores `tag_id`, never the tag name. Renaming "wisdom" to "insight" updates a single row and propagates automatically to every entry tagged with it.
- **Highlighted text** lives only in `highlights`. A linked journal entry stores `highlight_id`, not a copy of the selected passage.

D.4 Redundancy Summary

Nothing is duplicated across tables. If a user deletes a highlight, any linked journal entry survives with `highlight_id = NULL` (the entry had value of its own). If a user deletes their account, ownership cascades remove every piece of their private data in one operation.

E. UI–Database Mapping Table

Each major screen from Assignment 1 maps to specific tables and CRUD operations.

Screen	User Action	Table(s)	CRUD
Login / Register	Authenticate user	users	READ
Login / Register	Create new account	users, free_users, user_preferences, journals (freeform)	CREATE
Dashboard	Load "Continue Reading"	reading_progress, books	READ
Dashboard	Load recent journal entries	journal_entries, journals, books	READ
Dashboard	Load reading stats summary	reading_progress	READ
Library	Load user's book grid	books, reading_progress	READ
Library	Browse public domain	books (where is_public_domain=TRUE)	READ
Library	Upload a new book	books	CREATE
Library	Delete a book	books (cascades)	DELETE
Library	Search / filter books	books, collection_books, collections	READ
Library	Create a collection	collections	CREATE
Library	Add book to collection	collection_books	CREATE
Library	Remove book from collection	collection_books	DELETE
Library	Rename a collection	collections	UPDATE
Reader View	Open a book	reading_progress, books	READ
Reader View	Auto-save progress	reading_progress	UPDATE
Reader View	Highlight a passage	highlights	CREATE
Reader View	Highlight + write entry	highlights, journals (auto), journal_entries	CREATE
Reader View	Bookmark a page	bookmarks	CREATE
Reader View	Remove bookmark	bookmarks	DELETE
Reader View	Change highlight color	highlights	UPDATE
Reader View	Delete a highlight	highlights	DELETE
Reader View	Open book journal side panel	journal_entries, journals, highlights	READ
Journal Page	View all entries in a journal	journal_entries, journals	READ
Journal Page	Filter entries by tag	journal_entries, journal_entry_tags, tags	READ
Journal Page	Search entries by keyword	journal_entries	READ
Journal Page	Create a new entry	journal_entries	CREATE
Journal Page	Add tag(s) to entry	tags, journal_entry_tags	CREATE
Journal Page	Remove a tag	journal_entry_tags	DELETE
Journal Page	Edit entry title / content	journal_entries	UPDATE
Journal Page	Rename a journal	journals	UPDATE
Journal Page	Delete an entry	journal_entries	DELETE
Insights Dashboard	View stats (pages, streaks, hours)	reading_progress	READ
Customization	Load current preferences	user_preferences	READ
Customization	Change theme / font / sound	user_preferences	UPDATE
Profile & Settings	Update username / email	users	UPDATE
Profile & Settings	Change preferred language	users	UPDATE
Profile & Settings	Upgrade to paid	paid_users (INSERT), free_users (DELETE)	CREATE + DELETE
Profile & Settings	Delete account	users (cascades to all child tables)	DELETE

F. CRUD Definitions

CREATE — A new row is inserted when the user generates persistent data. Examples: **registering** (atomic insert into `users` → `free_users` → `user_preferences` → `freeform_journals`); **uploading a book** (`books`); **highlighting text** (`highlights`); **writing an entry** (insert into `journal_entries`, plus auto-insert of a `book_journals` row if the user hasn't journaled about this book before); **tagging** an entry (`tags` if new, then `journal_entry_tags`); **creating a collection** (`collections`). Maps to SQL `INSERT`.

READ — Existing data is retrieved throughout every session. **Login** reads `users`. The **dashboard** reads `reading_progress` joined with `books`. The **reader view** reads the book file plus all highlights, bookmarks, and linked journal entries for that page. The **journal page** reads entries filtered by journal, tag, or date. The **insights** dashboard aggregates from `reading_progress`. Maps to SQL `SELECT`, often with `JOIN`, `WHERE`, `ORDER BY`, or `GROUP BY`.

UPDATE — An existing row is modified. Examples: **reading progress** auto-saves every 30 seconds (updates `reading_progress.current_page`, `pages_read`, `total_seconds_read`, `last_read_at`); **editing a journal entry** updates its content; **changing a theme** updates `user_preferences.theme`; **renaming a collection** updates `collections.name`. Maps to SQL `UPDATE ... SET ... WHERE`.

DELETE — A row is removed. Examples: **removing a bookmark** (`bookmarks`); **deleting a journal entry** (cascades to `journal_entry_tags`); **deleting a highlight** (any linked entry survives with `highlight_id = NULL`); **removing a book from a collection** (deletes the `collection_books` link, the book itself stays); **deleting the account** (cascades through every owned table). Maps to SQL `DELETE FROM ... WHERE`.

G. Sample SQL Logic

Four examples per operation, all aligned with the schema in Section C.

G.1 SELECT

1. "Continue Reading" on the Dashboard

```
SELECT
  b.book_id,
  b.title,
  b.author,
  b.cover_image,
  b.total_pages,
  rp.current_page,
  rp.pages_read,
  ROUND((rp.pages_read / b.total_pages) * 100, 1) AS percent_complete,
  rp.last_read_at
FROM reading_progress rp
JOIN books b ON rp.book_id = b.book_id
WHERE rp.user_id = 7
  AND rp.is_completed = FALSE
ORDER BY rp.last_read_at DESC
LIMIT 5;
```

2. Load a book's journal, including the highlighted passage and tags for each entry

```

SELECT
    je.entry_id,
    je.title,
    je.content,
    je.entry_date,
    h.page_number,
    h.selected_text,
    GROUP_CONCAT(t.name ORDER BY t.name SEPARATOR ', ') AS tags
FROM journals j
JOIN journal_entries je      ON je.journal_id = j.journal_id
LEFT JOIN highlights h      ON je.highlight_id = h.highlight_id
LEFT JOIN journal_entry_tags jt ON jt.entry_id = je.entry_id
LEFT JOIN tags t            ON t.tag_id = jt.tag_id
WHERE j.user_id = 7
      AND j.book_id = 14
      AND j.type      = 'book'
GROUP BY je.entry_id
ORDER BY je.entry_date DESC;

```

3. Filter entries by tag across every journal the user has (Obsidian-style global tag)

```

SELECT
    je.entry_id,
    je.title,
    je.content,
    je.entry_date,
    j.type      AS journal_type,
    b.title     AS book_title
FROM journal_entries je
JOIN journal_entry_tags jt ON jt.entry_id = je.entry_id
JOIN tags t              ON t.tag_id    = jt.tag_id
JOIN journals j           ON j.journal_id = je.journal_id
LEFT JOIN books b        ON b.book_id   = j.book_id
WHERE t.user_id = 7
      AND t.name    = 'wisdom'
ORDER BY je.entry_date DESC;

```

4. Aggregate reading stats for the Insights Dashboard

```

SELECT
    COUNT(DISTINCT book_id)          AS books_started,
    SUM(CASE WHEN is_completed THEN 1 ELSE 0 END) AS books_completed,
    SUM(pages_read)                  AS total_pages_read,
    ROUND(SUM(total_seconds_read) / 3600.0, 1) AS total_hours_read,
    MAX(last_read_at)                AS last_session
FROM reading_progress
WHERE user_id = 7;

```

G.2 INSERT

1. Register a new user (creates users, free_users, user_preferences, and freeform journals atomically)

```

-- Step 1: base user row
INSERT INTO users (username, email, password_hash, preferred_lang)
VALUES ('layla_amin', 'layla@example.com', '$2b$12$hashedpw...', 'AR');

SET @new_user = LAST_INSERT_ID();

-- Step 2: ISA subclass row
INSERT INTO free_users (user_id) VALUES (@new_user);

-- Step 3: default preferences (1:1)
INSERT INTO user_preferences (user_id, theme, font, ambient_sound, sound_volume)
VALUES (@new_user, 'cozy-amber', 'Amiri', 'rain', 0.40);

-- Step 4: freeform journal (auto-created on signup)
INSERT INTO journals (user_id, book_id, type, title)
VALUES (@new_user, NULL, 'freeform', 'My Freeform Journal');

```

2. Save a highlight and link a new journal entry to it (book journal auto-created if missing)

```

-- Step 1: save the highlight
INSERT INTO highlights (user_id, book_id, page_number, selected_text, color)
VALUES (7, 14, 42,
        'So we beat on, boats against the current, borne back ceaselessly into the past.',
        'yellow');
SET @new_highlight = LAST_INSERT_ID();

-- Step 2: ensure the book journal exists (idempotent via UNIQUE constraint)
INSERT IGNORE INTO journals (user_id, book_id, type, title)
VALUES (7, 14, 'book', 'Notes on The Great Gatsby');

-- Step 3: look up the journal and add the entry
SET @book_journal = (
    SELECT journal_id FROM journals
    WHERE user_id = 7 AND book_id = 14 AND type = 'book'
);

INSERT INTO journal_entries (journal_id, highlight_id, title, content, entry_date)
VALUES (@book_journal, @new_highlight,
        'Reflections on the final line',
        'Fitzgerald closes the novel with this image of futility – Gatsby''s dream was always receding.',
        CURDATE());

```

3. Create a tag (if not present) and apply it to an entry

```

-- Create tag if the user doesn't have one by this name (INSERT IGNORE uses the UNIQUE(user_id, name) constraint)
INSERT IGNORE INTO tags (user_id, name) VALUES (7, 'wisdom');

-- Link tag to entry
INSERT INTO journal_entry_tags (entry_id, tag_id)
SELECT 89, tag_id FROM tags WHERE user_id = 7 AND name = 'wisdom';

```

4. Create a collection and add a book to it

```
INSERT INTO collections (user_id, name) VALUES (7, 'Classics');
SET @new_collection = LAST_INSERT_ID();

INSERT INTO collection_books (collection_id, book_id)
VALUES (@new_collection, 14);
```

G.3 UPDATE

1. Auto-save reading progress (called every 30 seconds while reading)

```
UPDATE reading_progress
SET
    current_page      = 67,
    pages_read        = pages_read + 3,
    total_seconds_read = total_seconds_read + 30,
    last_read_at      = NOW()
WHERE user_id = 7 AND book_id = 14;
```

2. Upgrade a user from free to paid (ISA transition)

```
-- Step 1: add row to paid_users
INSERT INTO paid_users (user_id) VALUES (7);

-- Step 2: remove row from free_users
DELETE FROM free_users WHERE user_id = 7;
```

3. Edit a journal entry's title and content

```
UPDATE journal_entries
SET
    title   = 'Reflections on the final line - revised',
    content = 'On rereading my notes, I think Fitzgerald meant this less as defeat and more as endurance...'
WHERE entry_id = 89;
```

4. Update customization preferences in one statement

```
UPDATE user_preferences
SET
    theme          = 'midnight-forest',
    background     = 'cabin-snow',
    ambient_sound  = 'fireplace',
    sound_volume   = 0.60
WHERE user_id = 7;
```

G.4 DELETE

1. Remove a bookmark

```
DELETE FROM bookmarks
WHERE bookmark_id = 23 AND user_id = 7;
```

2. Delete a journal entry (cascades through journal_entry_tags)

```
DELETE FROM journal_entries
WHERE entry_id = 89;
-- journal_entry_tags rows for this entry are auto-removed (ON DELETE CASCADE)
```

3. Delete a highlight (linked entries survive with highlight_id = NULL)

```
DELETE FROM highlights
WHERE highlight_id = 55 AND user_id = 7;
-- Any journal_entries with highlight_id = 55 now have highlight_id = NULL
-- (they become standalone entries in their journal).
```

4. Delete a user account (cascades through every owned table)

```
DELETE FROM users
WHERE user_id = 7;
-- Cascades clean up: free_users or paid_users, user_preferences,
-- reading_progress, highlights, bookmarks, journals (and their
-- journal_entries and journal_entry_tags), tags, collections
-- (and their collection_books). Books uploaded by this user
-- survive with uploaded_by = NULL (ON DELETE SET NULL).
```
